

```

import { q, one, j } from '../db.js';
import { getPackage } from './registry.js';

// A slot is a hole in the platform with a contract on it.
// The kernel declares the slots. Packages fill them. Nothing here names a vendor
const slots = new Map();

export function defineSlot({ key, summary, contract, multiple = false }) {
  slots.set(key, { key, summary, contract, multiple });
}

export function listSlots() { return [...slots.values()]; }

export async function persistSlots() {
  for (const s of slots.values()) {
    await q(
      `insert into provider_slot (key, summary, contract, multiple)
      values ($1,$2,$3::jsonb,$4)
      on conflict (key) do update set summary = excluded.summary,
      contract = excluded.contract, multiple = excluded.multiple`,
      [s.key, s.summary ?? null, JSON.stringify(s.contract), s.multiple]
    );
  }
}

// A provider package says which slot it fills. The exports it must supply come
// from the slot's contract, and are checked at load, not at 3am in production.
export function validateProvider(manifest, mod) {
  const slot = slots.get(manifest.fill);
  if (!slot) throw new Error(`${manifest.key} fills unknown slot "${manifest.fill}`);
  for (const fn of slot.contract.exports ?? []) {
    if (typeof mod[fn] !== 'function') {
      throw new Error(`${manifest.key} fills ${slot.key} but does not export ${fn}`);
    }
  }
  return slot;
}

// Swapping a provider is one row. No redeploy, no rebuild of anything above.
export async function bind({ businessId = null, slot, packageKey, config = {}, pr
  if (!slots.has(slot)) throw new Error(`no such slot: ${slot}`);
  const pkg = getPackage(packageKey);
  if (!pkg) throw new Error(`no such package: ${packageKey}`);
  if (pkg.manifest.fill !== slot) throw new Error(`${packageKey} does not fill $

```

```

if (!slots.get(slot).multiple) {
  await q(
    `update provider_binding set active = false
      where slot_key = $1 and business_id is not distinct from $2`,
    [slot, businessId]
  ).catch(async () => {
    await q(
      `update provider_binding set active = false
        where slot_key = $1 and (business_id = $2 or ($2 is null and business_i
          [slot, businessId]
        );
      });
  }

  return one(
    `insert into provider_binding (business_id, slot_key, package_key, config, pr
      values ($1,$2,$3,$4::jsonb,$5) returning *`,
    [businessId, slot, packageKey, JSON.stringify(config), priority]
  );
}

// A business binding wins over the platform default. That is the whole rule.
export async function resolve({ slot, businessId = null }) {
  const rows = await q(
    `select * from provider_binding
      where slot_key = $1 and active = true
        and (business_id = $2 or business_id is null)
        order by (business_id is not null) desc, priority asc`,
    [slot, businessId]
  );
  if (rows.length === 0) return null;
  const chosen = rows[0];
  const pkg = getPackage(chosen.package_key);
  if (!pkg) return null;
  return { binding: chosen, config: j(chosen.config) ?? {}, module: pkg.module, m
}

export async function resolveAll({ slot, businessId = null }) {
  const rows = await q(
    `select * from provider_binding
      where slot_key = $1 and active = true
        and (business_id = $2 or business_id is null)
        order by priority asc`,
    [slot, businessId]
  );
  // A business binding shadows the platform default for the same package,

```

```

// so a provider never appears twice with two different configs.
const seen = new Set();
const ordered = [...rows].sort((a, b) => (b.business_id ? 1 : 0) - (a.business_
const out = [];
for (const r of ordered) {
  if (seen.has(r.package_key)) continue;
  seen.add(r.package_key);
  const pkg = getPackage(r.package_key);
  if (pkg) out.push({ binding: r, config: j(r.config) ?? {}, module: pkg.module
}
return out.sort((a, b) => a.binding.priority - b.binding.priority);
}

export async function bindings(businessId = null) {
  return q(
    `select slot_key, package_key, business_id, priority, bound_at from provider_
      where active = true and (business_id = $1 or business_id is null)
      order by slot_key, (business_id is not null) desc`,
    [businessId]
  );
}

// --- the slots the kernel declares -----

defineSlot({
  key: 'workspace.executor',
  summary: 'Runs appmap steps against a device or environment.',
  contract: { exports: ['run', 'screenshot', 'prepare'] },
});

defineSlot({
  key: 'model',
  summary: 'Answers a prompt. Declares its models, their cost, speed and strength
  contract: { exports: ['models', 'complete'] },
  multiple: true,
});

defineSlot({
  key: 'harness',
  summary: 'Runs an agent loop, choosing capabilities until a goal is met.',
  contract: { exports: ['run'] },
  multiple: true,
});

defineSlot({
  key: 'builder',
  summary: 'Turns an intent into a plan composed of installed capabilities.',

```

```
    contract: { exports: ['plan'] },  
  });
```

```
defineSlot({  
  key: 'memory',  
  summary: 'Stores and recalls context across runs.',  
  contract: { exports: ['remember', 'recall'] },  
});
```

```
defineSlot({  
  key: 'sandbox',  
  summary: 'Runs an imported package in isolation before it is trusted.',  
  contract: { exports: ['test'] },  
});
```